

Simplifying User Interactions: A Large Action Model for web Automation

Friday 31st January, 2025 - 18:29

Abdeljalil Nabaoui
University of Luxembourg
Email: abdeljalil.nabaoui.001@student.uni.lu
ID: 022197615E

This report has been produced under the supervision of:

Salima Lamsiyah
University of Luxembourg
Email: salima.lamsiyah@uni.lu

Abstract

This project explores an automated system that interprets user instructions in natural language and transforms them into executable Python Selenium code. Initially, the goal was to automate various websites, but due to technical and compatibility complications including unpredictable site structures and rapidly changing page elements, the scope of the project was narrowed to a single target platform: YouTube. By focusing on YouTube, we aimed to create a robust pipeline for breaking down general user tasks (e.g., searching and clicking on videos) into concise, step-by-step instructions. In parallel, a language model generates Selenium code that attempts to complete each step. If execution fails or code updates are needed, the system receives user feedback, attempts multiple fix strategies, and reruns the corrected snippet. Despite the reduced scope, our approach highlights how AI can unify instruction parsing, code generation, and iterative error handling to create a more user-friendly automation workflow.

code generation for YouTube automation. Our key contributions include

- **Step-by-Step Instruction Generation:** We design a pipeline that takes a high-level user query (e.g., “Search for a funny compilation on YouTube”) and splits it into concise instructions, each focusing on a single actionable command.
- **Iterative Code Fixes and Error Handling:** When a generated code snippet fails to run or needs an update, the system accepts user feedback, regenerates the snippet with different strategies, and reruns the corrected version.

1. Introduction

Artificial Intelligence (AI) and Machine Learning (ML) have seen rapid advancements in recent years, inspiring researchers and practitioners to develop more intuitive, human-centric tools. One promising avenue is enabling computers to understand natural language instructions and convert them into actionable, context-aware code. This can reduce the barrier to automating tasks that traditionally require manual programming expertise.

Our initial ambition was to automate a variety of websites, from simple to complex structures, through input from natural language. However, difficulties such as inconsistent site layouts, frequently changing selectors, and challenging page dynamics led us to reduce the project’s scope to YouTube. By selecting a single, stable platform, we avoid entanglements with diverse web elements and focus on designing and refining an AI-driven instruction processing and code synthesis pipeline. In this way, we highlight an approach that remains valid even if restricted to a single popular site.

In this paper, we present a browser automation system that relies on AI-driven parsing of user instructions and subsequent

This paper is structured as follows.

- 1) *Abstract* – Summarizes the objective and contributions of our system.
- 2) *Introduction* – Motivates the work, explains the reduction in scope to YouTube, and outlines our contributions.
- 3) *Related Work* – Examines existing approaches in language-to-code systems and browser automation, highlighting where our approach fits.
- 4) *Scientific Background* – Describes the relevant concepts for natural language understanding and Selenium-based automation, focusing on YouTube context.
- 5) *Methodology* – Details how instructions are generated, code snippets are produced, and errors are fixed in a multistep pipeline.
- 6) *Experimental Results* – Showcases performance metrics using YouTube tasks, evaluation methods, and demonstrations.
- 7) *Conclusion and Future Work* – Summarizes the project’s achievements and limitations while proposing how the approach will be extended to more diverse websites in the future.

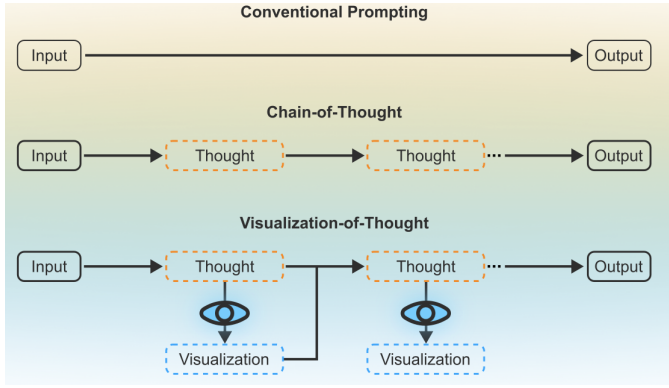


Fig. 1: Comparison of Prompting Methods: Conventional, Chain-of-Thought, and Visualization-of-Thought for Enhanced Prompt Flow.

2. Related Work

Recent progress in large language models (LLMs) and their ability to handle complex instructions can be seen in multiple fields, including spatial reasoning, zero-shot prompting, and multimodal interfaces. One focal area is *spatial reasoning*, in which LLMs learn to interpret positional and geometric relationships. In [Wenshan Wu et al.(2024)], the authors introduce a "visualization of thought" procedure, encouraging the model to develop internal imagery that aids in comprehending and executing spatial commands. Although our system emphasizes website-driven tasks rather than pure geometry, the underlying principle, transforming abstract text into structural or layout-based understanding, guides how an LLM may parse and navigate web pages.

Another key element is *zero-shot prompting*, in which the model tackles new problems without domain-specific examples or exhaustive training. This is especially relevant to code generation scenarios: In [Yann LeCun(2022)], researchers discuss directing large language models toward tasks in an autonomous fashion, bypassing the need for meticulously annotated datasets. For our objective, Web automation on YouTube, using a largely untrained approach is attractive, since the model can operate with minimal site specific knowledge while still delivering workable automation code.

Alongside academic literature, there is also exploration of how humans use visuals to enhance their understanding of texts and narratives. A video on visual learning [Scriberia (2017)] explains how humans form mental pictures or "storyboards" when processing narratives. For example, when reading a storybook, readers often imagine scenarios and visuals in their mind to better grasp the plot and characters. Although this concept is not related to code-based tasks, it underscores the importance of visualization in following instructions and understanding complex scenarios. By analogy, large language models could benefit from mechanisms that mimic this human ability to conceptualize tasks through a visual or structured representation of a website or task layout.

Suhr and Lewis [Alane et al.(2017)] contributed a corpus

for natural language visual reasoning, showing how text can reference objects and relationships that require a model to go beyond superficial interpretation. Jankoski and Gievska [Kristijan et al.(2018)] evaluate multiple solutions for visual question answering, further indicating that bridging language with contextual or layout-based clues improves the performance of a model. In line with this, our system must adapt from high-level instructions to a more contextualized browser environment.

Finally, community-driven projects like Pywinassistant [Pywinassistant project] illustrate how single-step instructions or short command sequences can be turned into automated scripts, although primarily for the Windows GUI. Our approach aims to similarly harness large language models but for browser-based tasks involving user defined objectives on a single site: YouTube. By focusing on a narrower domain, we can study how the internal reasoning of an LLM augmented with zero-shot prompting and iterative fix attempts achieves more robust instruction parsing and code generation performance.

3. Scientific Background

Building an automated system that translates natural language instructions into executable code requires a careful interaction of machine learning methods, language-model prompting techniques, and robust programming frameworks for web interaction. In this section, we discuss the main scientific concepts that underpin the implementation of our project.

3.1. Large Language Models and Prompting Strategies

At the heart of our system is a large language model (LLM) capable of receiving an instruction in plain text and returning Python code tailored to a specific browser automation task (in our case, interacting with YouTube). LLMs typically rely on transformer based architectures to handle contextual embeddings of words, sub-words, or tokens. This encoding allows them to capture both short and long range dependencies in a sentence.

3.1.0.1. Zero-Shot Prompting. Rather than providing domain-specific examples for every website or element of the page, our system leverages zero-shot prompts. In this paradigm, the model receives a brief textual description (*prompt*) that explains the desired task, but does not include curated example-label pairs. The LLM then produces a best-guess output. In our application, that means the model must infer how to generate a Selenium snippet, e.g. using CSS selectors or XPath from short instructions such as 'Click on the first video in the search results'.

3.2. Adaptive Code Generation and Iterative Fixes

While large language models can produce code aligned with user instructions, their output is not always guaranteed to be

up to date / work as intended. Thus, a *iterative fix* mechanism is introduced. After running an LLM script, the system checks for errors. If errors occur, it prompts the user for a brief textual explanation of the issue (e.g. 'Use a different CSS selector' or 'wait for the element to load'). The model then modifies only the snippet for the *current* instruction, rather than rewriting the entire program.

This loop draws on reinforcement principles where the user's correction acts as feedback. However, rather than a strictly numerical reward, we redirect textual 'fix instructions' back into the model so that subsequent code addresses the specific error message or environment discrepancy that arises; in other words, the user trains the LLM by using textual feedback which is interactive.

3.3. Selenium WebDriver and DOM Interaction

Our pipeline uses Python-based Selenium for browser automation. Selenium WebDriver interacts with the Document Object Model (DOM) to locate and manipulate page elements. Typically, an instruction such as "Search for "funny compilation" in the YouTube search bar" translates into code that:

- 1) Finds the search bar element
(`driver.find_element(By.NAME, "search_query")`)
- 2) Sends keystrokes (e.g., "funny compilation")
- 3) Presses Enter (`Keys.RETURN`) to initiate the search

This direct mapping from user instructions to DOM-based actions is made possible by bridging an LLM's text generation capabilities with Selenium's well-defined methods for page interaction.

3.4. Execution Environment and Error Handling

3.4.0.1. Execution of Generated Code. Once the LLM produces a code snippet, we run it in a Python environment that has access to the `selenium` library and a live browser session. To prevent multiple browser windows from opening, the system omits any code line that re-initializes `webdriver.Chrome()` or closes it.

3.4.0.2. Multi-Fix Approach. If the code snippet fails (selects an outdated or incorrect CSS selector), our system requests user-provided fix instructions. The model then generates a refined snippet. We rerun only that snippet while retaining the overall environment state, allowing immediate verification of whether the fix resolves the error. A limit on fix attempts (three attempts) prevents us from getting stuck in an unproductive loop, therefore if this multi-fix approach does not work the next step is to tweak / engineer the prompt for a better result.

3.5. Reduced Scope to a Single Website: YouTube

While the conceptual approach could theoretically target any site, the system is currently restricted to YouTube for

practical reasons (costs of the LLM used, etc.). YouTube offers a moderately consistent structure, allowing us to highlight the model's capacity to think about page elements (search boxes, clickable video thumbnails, etc.) without having to contend with the wide-ranging site structures present on other websites.

In general, these scientific components; LLM-based zero-shot prompting, Selenium-based automation, and iterative fix instructions, form the backbone of our pipeline that converts human language instructions into browser automation steps. Each step is processed individually to minimize cross-instruction interference and reduce the chances of re-initializing a browser session.

4. Methodology

Our system follows a pipeline approach in which a user-provided prompt is broken down into actionable steps by a large language model, and each step is then converted into executable Selenium code. Figure 2 illustrates the primary modules of this pipeline:

- 1) **User Prompt:** The user inputs a high-level instruction (e.g. "Go to YouTube and search for a funny compilation"). This prompt captures the overall goal, but may not specify low-level details.
- 2) **Instruction Generation (Mistral LLM):** The Mistral language model takes the prompt and, based on any available page or state information, produces a single actionable *Step*. If the original user goal has not yet been achieved, the model continues to generate new instructions in subsequent rounds. (Refrence for mistral)
- 3) **Code Generator (Selenium):** Each *Step* is passed into a code generator that formulates a Python snippet containing Selenium commands. Crucially, this snippet *must not* open a new instance of the browser or merges the code of previous instructions. Instead, it focuses solely on the current action (e.g., clicking on an element).
- 4) **Execution:** The generated code runs in the live browser session where the user can examine the behavior. After the execution of each step, the system requests user feedback to either continue to the next step if the current one is successful or to provide feedback if the code fails or produces undesired outcomes (e.g., incorrect CSS selectors or page timing issues).
- 5) **State (Vision):** Meanwhile, the system maintains an internal representation of the webpage; using a screenshot-based approach, to help the language model decide the next best action. Once the sub-goal is deemed accomplished, new steps are either generated or if the final goal is met the user is required type 'finish' in the feedback box which will end the process.

Step-by-Step Flow:

- 1) *Receive Prompt.* The user types a general objective. For instance, "Search for a funny compilation on YouTube and watch the first video."
- 2) *Produce Step.* The Mistral LLM parses the user request and outputs a single, specific step (e.g., "Open

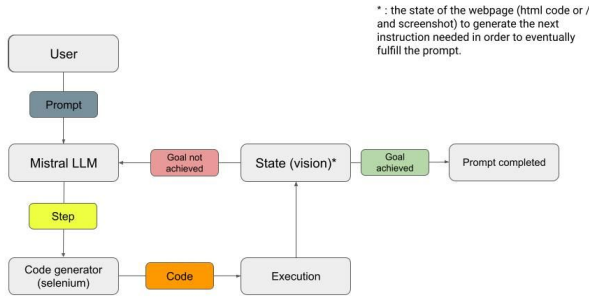


Fig. 2: High-level Architecture of the System. The system utilizes the Mistral LLM to generate a single step based on the user’s prompt and the current state of the task. This step is then passed to an OpenAI model (currently ChatGPT 4o, with the possibility to switch to a more advanced model if required) to produce Selenium code corresponding to the step. The generated code is executed in a live browser session, and feedback from the task’s state (e.g., webpage HTML or visual snapshot) guides subsequent iterations until the prompt is successfully completed.

YouTube”), rather than attempting the entire sequence in one shot.

- 3) *Generate Code*. A code generator (ChatGPT 4o) converts the step into Python Selenium code:

```
driver.get("https://www.youtube.com")
```

- 4) *Execute Snippet*. The newly generated code is executed in the same, ongoing browser session. This approach ensures that we do not open multiple instances or inadvertently restart the browser.
- 5) *Check Outcome*. If the snippet fails (e.g., element not found), the system asks the user for fix instructions (e.g., “Accept the pop-up first”), regenerates only the relevant snippet, and attempts execution again.
- 6) *Update State*. Whenever a substep is performed, the system updates its “vision” or knowledge of the state of the web page. The language model uses that updated information in the next round to determine if the final goal has been achieved or if new steps are needed.
- 7) *Repeat or Finish*. If the overarching goal is not achieved, the process is retriggered to produce a new step. If all tasks have been completed (e.g., the video starts playing), using user feedback, the process is finished. All of the steps, code snippets and errors are viewable by the user so the user can give better feedback and make the system more interactive.

In essence, the methodology enforces a strict *one instruction, one snippet* rule to ensure clarity in code generation and prevent cross-step code contamination. The user only provides fix instructions when something goes wrong, allowing for robust and iterative refinement in real time.

5. Experimental Results

Task 1 = ‘Go to YouTube, search for a funny compilation and watch the first video.’

TABLE 1: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Success in 3 steps	Baseline performance 36 seconds to complete 0 errors
2	Success in 3 steps	32 seconds to complete 0 errors
3	Success in 3 steps	27 seconds to complete 0 errors
4	Success in 3 steps	42 seconds to complete Plays the video in preview mode instead of opening it in a different page unlike the other trials

Task 2 = ‘Go to YouTube, search for ‘motivational speech’, play the first video, and scroll to the comment section’

TABLE 2: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Success in 3 steps	Baseline performance 43 seconds to complete
2	Failure	The first video in the results in an ad so it tried to select it but it’s a website 83 seconds
3	Success in 3 steps	0 errors 29 seconds to complete 2 errors 120 seconds to complete
4	Partial success	Same problem with 2 trial but with user feedback explaining the problem it selects the second video and goes to the comment section but adds another step and tries to comment

Task 3 = ‘Go to YouTube, go to ‘NASA’ official channel, and play their latest video.’

Task 4 = ‘Go to YouTube, search for ‘Lo-Fi Chill Beats playlist’, and start playing the first playlist.’

Task 5 = ‘Go to YouTube, search for ‘fitness tips’, select a channel, and subscribe.’

Task 6 = ‘Go to YouTube, search for ‘travel vlogs’, play the first video, and skip the ad if one appears.’

6. Conclusion and Future Work

The development of a Large Action Model (LAM) capable of automating YouTube interactions marks a significant step

TABLE 3: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Failure	61 seconds Error in generating step: 500 "error": "Model too busy, unable to get response in less than 60 second(s)" which highlights a problem where the mistral model can sometimes be too busy
2	Failure	208 seconds It searches for NASA's channel but can't select the channel even with user feedback
3	Failure	317 seconds goes to the NASA channel but can't get the selector for the videos tab correctly so it hallucinates extra steps
4	Success in 6 steps	3 errors 135 seconds to complete Can't find the selector for the videos tab so i instructed it using feedback to select the latest video (second search result) in the search results for 'NASA'

TABLE 4: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Failed	Couldn't distinguish between a normal video and a playlist
2	Success in 4 steps	0 errors 40 seconds to complete
3	Success in 4 steps	0 errors 37 seconds to complete
4	Success in 3 steps	0 errors 43 seconds to complete

toward creating systems that bridge natural language understanding and web-based automation. This project successfully integrated the Mistral model for generating task instructions and OpenAI's ChatGPT 4o for converting these instructions into Selenium executable code. By iteratively using the state feedback from the website to refine the generated steps, the system demonstrated its ability to dynamically adapt to the user's request.

Despite these achievements, the project faced notable challenges. One major limitation was the restricted scope to YouTube automation due to the complexity of generalizing across various websites. Factors such as inconsistent webpage structures, dynamic content loading, and varying site functionalities introduced hurdles to creating a robust and versatile solution. Furthermore, the reliance on Selenium for task execution posed challenges in ensuring reliable interaction with web elements, particularly on highly dynamic or heavily scripted web pages.

Looking ahead, there is significant potential to expand the scope of the project to encompass a wider range of websites. This would involve implementing advanced techniques for generalizing state extraction, such as utilizing computer vision models to analyze visual webpage layouts or improving the NLP model's ability to infer task-specific nuances from prompts. Furthermore, prompt engineering and more experiments are needed to improve the quality and accuracy of the

TABLE 5: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Success in 5 steps	Selects the channel after going into a video 1 errors
2	Success in 6 steps	60 seconds to complete, had to correct it because it went to a video which is not fitness related
3	Success in 5 steps	0 errors 37 seconds to complete
4	Success in 4 steps	1 errors 42 seconds to complete Tries to subscribe but generate the wrong code

TABLE 6: Trial Outcomes and Insights

Trial	Outcome Variance	Insights Gained
1	Failed	Tried to skip the video too early and started hallucinating 2 errors
2	Success in 5 steps	41 seconds to complete after telling it to wait for the ad skip button to appear using human feedback
3	Failed	Same problem as Trial number 1
4	Success in 4 steps	0 errors 43 seconds to complete

steps and code generated.

Another key avenue for future work lies in optimizing the system for performance and scalability. This includes exploring lightweight alternatives to Selenium, such as Playwright, to reduce execution time and increase reliability. Integrating support for multiple languages and cultural contexts would make the system more inclusive.

In conclusion, this project represents a promising foundation for automating web-based interactions through natural language. By addressing the identified limitations and extending its capabilities, the proposed LAM system has the potential to become a powerful tool for automating tasks across the Internet, ultimately bridging the gap between human language understanding and practical web automation.

References

- [Wenshan Wu et al.(2024)] Wenshan Wu and Shaoguang Mao and Yadong Zhang and Yan Xia and Li Dong and Lei Cui and Furu Wei, "Mind's Eye of LLMs: Visualization-of-Thought Elicits Spatial Reasoning in Large Language Models" *arXiv preprint arXiv:2404.03622*, 2024.
- [Yann LeCun(2022)] Yann LeCun, "A Path Towards Autonomous Machine Intelligence" *OpenReview Proceedings*, 2023, <https://openreview.net/pdf?id=BZ5a1r-kVsf>, accessed May 2024.
- [Scriberia (2017)] Visual Learning Explained, "Video Explanation of how we use Visual Learning in our daily life" <https://www.youtube.com/watch?v=rZikhQ5w5Ck>, accessed May 2024.
- [Alane et al.(2017)] Alane Suhr, Mike Lewis, James Yeh, Yoav Artzi, "A Corpus of Natural Language for Visual Reasoning" *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017.
- [Kristijan et al.(2018)] K. Jankoski and S. Gievska, "Evaluation of Multiple Approaches for Visual Question Reasoning" *Conference on Future Technologies for Collaborative Research (FTCR)*, 2018.
- [Pywinassistant project] A. Real, "Pywinassistant," <https://github.com/a-real-ai/pywinassistant>, accessed May 2024.